Reasoning Genome Chain: A DNA-Inspired Control Layer for Auditable Self-Evolving AI Inference

Technical Report v0.1

Yang Hao (杨浩)

June 26, 2026

Repository: https://github.com/yanghao1143/rust-norion Gitee mirror: https://gitee.com/babalibaba/rust-norion Zenodo DOI: https://doi.org/10.5281/zenodo.20901489 OSF archive: https://osf.io/cybdm/ ScienceDB DOI: https://doi.org/10.57760/sciencedb.41287 OpenI project: https://openi.pcl.ac.cn/asd8841315/rust-norion

Abstract

Large language model applications increasingly depend on control logic outside the model weights: memory retrieval, routing, tool use, reflection, evaluation, rollback, and operator approval. In many prototypes this control logic is implicit, scattered across prompts, scripts, logs, and ad hoc agent state. This technical report introduces the Reasoning Genome Chain, a DNA-inspired software-control abstraction implemented in the open-source rust-norion prototype. The approach does not claim biological simulation and does not retrain model weights. Instead, it represents reusable reasoning behavior as bounded, typed, auditable strategy records called reasoning genes. A task profile selects an express chain that can influence runtime routing, retrieval, reflection, budget posture, tool dispatch, and validation gates, while a separate memory chain preserves provenance, fitness evidence, rejection reasons, rollback anchors, and privacy-safe digests. Gene Scissors provides a guarded mutation pipeline for relabel, cut, splice, quarantine, repair, crossover, rollback, and regenerate operations. Durable mutation is denied by default and must pass trace, test, benchmark, drift, privacy, license, rollback, and operator-approval gates before admission. The report describes the architecture, schema, safety model, prototype surfaces, local validation strategy, open publication artifacts, and an update automation path for keeping dataset descriptions and outreach material synchronized with repository changes. The contribution is a concrete engineering frame for building auditable self-evolving inference control layers in Rust, with explicit separation between runtime expression, append-only evidence, and write authorization.

1. Motivation

Modern AI systems are no longer only prompt-to-answer calls. Useful systems retrieve context, route tasks, call tools, inspect partial answers, replay past experience, enforce safety boundaries, and decide whether new experience should be reused. These behaviors form a control layer around inference. If the control layer is implicit, the system becomes difficult to debug: a successful answer may not reveal which strategy helped, a failed answer may not provide a rollback target, and a self-improvement loop may silently preserve polluted state.

rust-norion starts from a simple premise: the layer around inference should be made explicit enough to test, inspect, and reverse. The project uses Rust because the problem is largely one of state, boundaries, evidence, and failure paths. Types, deterministic tests, append-only records, and conservative writer gates are more useful here than a larger prompt template.

The DNA metaphor in this report is intentionally narrow. A biological genome is not being simulated. Instead, the metaphor names a software architecture: reasoning behavior can be segmented, labelled, expressed, evaluated, isolated, recombined, and rolled back under strict evidence gates.

2. Scope and Non-Claims

This report describes a research-engineering prototype, not a production LLM inference kernel. It also does not claim that rust-norion has produced a state-of-the-art model, a new biological model, or a complete autonomous self-improving system.

The current scope is:

raw private prompts, hidden reasoning, model weights, or secrets;

- a typed vocabulary for DNA-inspired reasoning-control records;
- a dual-chain architecture for runtime expression and evidence provenance;
- a preview-first mutation pipeline for controlled strategy edits;
- trace and benchmark evidence surfaces for safety checks;
- local-first Rust implementation surfaces that can be tested without exposing
- contributor and publication infrastructure for open review.

The most important limitation is that v0.1 is primarily an architectural and prototype report. Strong empirical claims require future benchmark suites, ablation studies, and external reproduction.

3. Contributions

This report makes five practical contributions for open review:

boundary instead of treating it as prompt glue.

traces, mutation plans, and rollback anchors as inspectable software records.

relabel, cut, splice, quarantine, repair, crossover, rollback, and regenerate intents remain blocked until evidence gates pass.

model weights, hidden reasoning, and private trace payloads out of durable genome records.

can review the paper, reproduce the repository state, and keep public dataset descriptions aligned with ongoing changes.

1 It names the model-external control layer as a first-class Rust system

2 It defines reasoning genes, express chains, memory chains, expression

3 It introduces Gene Scissors as a preview-first mutation workflow where

4 It describes a fail-closed safety model that keeps raw prompts, secrets,

5 It publishes citable artifacts and automated update packets so contributors

4. System Overview

The Reasoning Genome Chain treats a reasoning-control policy as a set of bounded strategy records.

Figure 1 summarizes the control flow.

```
 Task profile
     |
     v
 ReasoningGenome
     |
     +--> express_chain ----> GenomeExpression ----> inference control
     |       |                    |                    routing, memory,
     |       |                    |                    reflection, tools
     |       v                    v
     |   ExpressionTrace ----> evidence gates ----> admit / reject / rollback
     |
     +--> memory_chain -----> provenance ledger
             |                 source ids, digests, fitness, drift,
             |                 rejection reasons, rollback anchors
             v
```

```
Gene Scissors preview
relabel / cut / splice / quarantine / repair / crossover /
rollback / regenerate
```

4.1 ReasoningGene

A `ReasoningGene` is one strategy atom. Examples include:

- a memory retrieval posture;
- a route-threshold bias;
- a hierarchy-balance preference;
- a reflection checklist;
- a language-mode preference;
- a Rust-only tool policy;
- a sub-agent budget posture;
- a safety constraint.

Each gene carries identifiers, labels, purpose tags, age/freshness metadata, fitness evidence, trust and drift scores, lineage, source-evidence identifiers, and a rollback anchor. It does not carry raw private prompt payloads.

4.2 ReasoningGenome

A `ReasoningGenome` is an ordered chain or small graph of genes selected for a task profile. A Rust coding profile can prefer compiler evidence, focused tests, and review gates, while a Chinese writing profile can prefer bilingual reflection and gist memory. Profiles keep local strategy genomes from overwriting one global heuristic.

4.3 GenomeExpression

`GenomeExpression` is the runtime projection of the selected genome into an inference request. It may influence router thresholds, retrieval limits, hierarchy weights, reflection checks, tool plans, agent-team hints, budget posture, and replay priority.

4.4 ExpressionTrace

`ExpressionTrace` records sanitized evidence of what the genome changed during a run: active gene ids, profile, route-budget deltas, memory-policy deltas, validation gates, reward inputs, rollback eligibility, and mutation-plan status. Trace output is designed to explain strategy effects without exposing hidden reasoning or raw private data.

5. Dual-Chain Architecture

The DNA double-strand inspiration maps to two software chains.

5.1 Express Chain

The `express_chain` is small, runtime-visible, and task-facing. It contains the active genes that can influence routing, memory retrieval, reflection, tool dispatch, and budget posture for the current task. It is optimized for fast projection during inference.

5.2 Memory Chain

The `memory_chain` is larger and append-only. It preserves where a gene came from and why it should or should not be reused: stable anchors, source experience ids, KV/gist memory ids, fitness summaries, drift evidence, validation gates, rejection reasons, and rollback links.

The split has two design benefits. First, runtime expression stays small enough to inspect and project. Second, provenance can remain durable without loading raw private prompts, .ndkv payloads, secrets, copied third-party internals, or hidden reasoning into active control state.

6. Gene Scissors: Guarded Mutation

Gene Scissors is the controlled editor for the Reasoning Genome Chain. It may propose edits, but it cannot bypass validation or mutate private runtime state directly. Supported edit intents are:

operator-approved proposal;

memory admission, repeated test failure, or excessive compute waste;

dry-run gate before admission;

validated siblings, and clean replay evidence after quarantine.

- relabel: refresh a stale gene label, purpose, or tag set;
- cut: remove or disable a low-fitness gene from a profile-specific chain;
- splice: insert a validated gene from a clean experiment, replay run, or
- quarantine: isolate a gene linked to drift, prompt leakage risk, unsafe
- repair: replace a malformed gene reference with a known safe fallback;
- crossover: combine compatible genes from high-fitness chains, then force a
- rollback: restore the previous stable genome when a chain regresses;
- regenerate: rebuild a replacement gene from a stable rollback anchor,

Every durable edit must carry a MutationPlan with changed gene ids, source evidence ids, expected phenotype, validation commands, rollback target, and admission state. Preview mode remains read-only.

7. Aging, Malignancy, and Rejuvenation

The "aging" vocabulary refers to software freshness. A useful gene can become stale when its label no longer matches its current purpose, when its evidence is old, or when task profiles change. The correct first action is often not deletion but a read-only relabel plan.

A malignant gene is treated as contaminated strategy, not something to repair in place. Malignancy can be triggered by repeated drift, unsafe memory admission, privacy risk, high contradiction pressure, benchmark regression, or repeated compiler/test failure. The safe sequence is:

siblings;

license, and operator-approval gates pass.

1 detect malformed behavior with bounded evidence;

2 quarantine the gene so it cannot influence expression;

3 cut or disable the contaminated strategy only after a rollback anchor exists;

4 regenerate a replacement from stable anchors and validated high-fitness

5 admit the replacement only after trace, tests, benchmark, drift, privacy,

The prototype includes a genome-rejuvenation simulation surface that covers healthy genes, stale labels, low-fitness routing genes, and malignant safety genes. The report remains digest-only and read-only.

8. Safety and Privacy Model

The control layer follows fail-closed defaults:

not copied into genome records;

`operator_approval_required = true`, `admission_write_authorized = false`, and `applied = false` are preview defaults;

privacy/license checks, and explicit maintainer or operator approval;

team coordination, or adaptive state must pass trace/schema gates before persistence;

- raw private prompts, raw chat logs, model weights, and `.ndkv` payloads are
- records use ids, summaries, counters, bounded metrics, and digests;
- `read_only = true`, `write_allowed = false`,
- durable writes require writer gates, validation evidence, rollback plans,
- any proposal that touches routing, memory, reflection, tool dispatch, agent
- rollback is part of the edit plan, not an afterthought.

This safety model is intentionally conservative. It makes self-evolution slower than direct mutation, but it prevents a prototype from silently turning feedback into durable polluted state.

9. Implementation Surfaces in rust-norion

The local repository implements the report as a set of Rust and documentation surfaces rather than a single monolithic service. The main surfaces include:

`MutFixer` semantics;

governance, runbooks, and research;

future iteration-update drafts;

and English update posts, and a JSON manifest from recent repository changes without requiring external API keys.

- architecture documents for Reasoning Genome Chain and schema terms;
- read-only splicing previews through `dna_splicer`, `MutDetector`, and
- digest-only trace and mutation-plan evidence;
- benchmark-gate surfaces for genome expression and rejuvenation;
- contributor lanes for control layer, memory, runtime boundary, benchmark,
- outreach automation that tracks relevant Rust/AI communities and generates
- publication-update automation that generates dataset descriptions, Chinese

As of this report, the community registry validates successfully with 463 tracked communities, four outreach templates, 300 submitted external targets, 10 manual-login or verification targets, 153 deferred or waiting targets, and seven iteration-update candidates. These outreach numbers are not scientific evaluation metrics; they are open-source dissemination evidence.

10. Validation Strategy

v0.1 validation is framed as engineering evidence rather than benchmark superiority. The recommended local validation ladder is:

rejuvenation decisions, rollback readiness, and digest-only ledger output;

findings, proposals, and read-only status appear in control-plane evidence;

readiness, no durable writes, and non-regression of memory usefulness;

third-party internals from entering durable records;

1. schema inspection for reasoning-genome records and preview defaults;
2. focused Rust tests for genome expression, mutation-plan construction,
3. trace checks that ensure splice segments, exon/intron/variant labels,
4. benchmark gates requiring decision-kind coverage, replay digests, rollback
5. privacy and clean-room checks that prevent raw private payloads or copied
6. community registry validation for reproducible outreach metadata.

The current branch has also passed the repository's outreach validation workflow and focused Rust crate validation on GitHub Pull Request #206.

11. Publication And Update Automation

The project deliberately separates artifact generation from external submission. The repository can automatically produce update packets from recent commits, changed files, and stable publication links. Those packets are safe to use as draft material for ScienceDB descriptions, OSF project updates, GitHub Release notes, CSDN articles, Rust community posts, and OpenI project updates.

External platform submission remains manual because accounts, author identity, institutional email, category selection, moderation terms, and CAPTCHA or SMS checks belong to the author and platform. Automation therefore stops at a reviewable packet: dataset description, Chinese post, English post, and JSON manifest. This keeps the public record synchronized with repository changes without pretending to bypass platform governance.

The update generator is:

```
./tools/outreach/generate-publication-update.ps1 -SinceDays 7 -MaxCommits 40 -OutDir publication-update
```

It is wired into GitHub Actions as a scheduled, push-triggered, and manually dispatchable workflow. The output can be attached to a new dataset version, copied into a preprint platform update, or used as source material for a contributor-facing article.

12. Relation to Existing Work

The Reasoning Genome Chain is related to several lines of work. ReAct combines reasoning traces and actions so language models can interact with external environments while maintaining task progress. Reflexion reinforces language agents with verbal feedback and episodic memory instead of direct weight updates. Toolformer studies self-supervised tool-use decisions inside language models. SWE-agent highlights that language-model agents benefit from interfaces designed for their capabilities and constraints.

rust-norion differs in focus. It is not a new prompting pattern or an agent-benchmark result. It is a software-control proposal for representing, auditing, and gating the strategies around inference. Its central questions are: what changed in the control layer, what evidence supports the change, what rollback anchor exists, and who authorized durable admission?

The project also uses biology-inspired references only as conceptual analogies. Public papers and repositories are research references, not source-code templates. Implementation migration must be by behavior specification, tests, license review, and clean-room boundaries.

13. Open Research Questions

Several questions remain open:

quality?

results, latency, wasted-compute reduction, memory usefulness, user feedback, and drift severity?

reasoning or private data?

which require human approval forever?

without coupling the project to one provider?

- What benchmark tasks best isolate control-layer value from underlying model
- How should gene fitness combine reflection diagnostics, compiler/test
- What is the minimal trace schema that remains useful without exposing hidden
- Which mutation intents are safe to automate under preview-only rules, and
- How should multi-agent proposals be merged without writer conflicts?
- What evidence is strong enough to admit a regenerated gene after quarantine?
- Can dual-chain control records improve reproducibility across model backends

14. Roadmap

The near-term roadmap is:

the project evolves;

email requirements are satisfied;

iteration-update automation.

- publish this technical report as a citable artifact;
- keep the Zenodo, OSF, ScienceDB, GitHub Release, and OpenI records aligned as
- continue ChinaXiv and SinoXiv submission after author-login and institutional
- expand experiments from schema and safety gates to comparative benchmarks;
- add diagrams and reproducible examples for each contributor lane;
- improve docs.rs/crates.io readiness for publishable Rust crates;
- continue community outreach through monthly and release-driven

15. Conclusion

The Reasoning Genome Chain reframes AI self-evolution as a software-control problem. Instead of mutating weights or accumulating unstructured prompt history, rust-norion records strategy atoms as auditable genes, expresses them through a small runtime chain, preserves provenance in an append-only memory chain, and forces mutation through preview-first evidence gates. The result is not a finished autonomous system, but a concrete Rust prototype for making the layer around inference explicit, testable, local-first, and reversible.

Availability

Source code and documentation are available at:

- GitHub: https://github.com/yanghao1143/rust-norion
- Gitee mirror: https://gitee.com/babalibaba/rust-norion
- GitHub Release: https://github.com/yanghao1143/rust-norion/releases/tag/rgc-v0.1.0
- Zenodo DOI: https://doi.org/10.5281/zenodo.20901489
- OSF archive: https://osf.io/cybdm/

- ScienceDB DOI: https://doi.org/10.57760/sciencedb.41287
- ScienceDB CSTR: https://cstr.cn/31253.11.sciencedb.41287
- OpenI project: https://openi.pcl.ac.cn/asd8841315/rust-norion

Ethics, Safety, and Governance Statement

This project treats self-evolution as a guarded engineering workflow. It does not attempt to hide autonomous state mutation. Preview is read-only by default; durable mutation requires explicit evidence and approval. The project also keeps a clean-room boundary for external references and avoids storing raw private prompts, secrets, hidden reasoning, copied third-party internals, or raw model payloads in genome records.

Acknowledgments

This report is based on the rust-norion open-source prototype and its contributor-facing architecture, governance, and outreach documents. External agent and tool-use papers are cited as conceptual context, not as source-code inputs.

References

Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629

Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. https://arxiv.org/abs/2303.11366

Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023. https://arxiv.org/abs/2302.04761

Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793, 2024. https://arxiv.org/abs/2405.15793

Repository documentation, 2026. https://github.com/yanghao1143/rust-norion

1 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik

2 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik

3 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli,

4 John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao,

5 rust-norion contributors. Reasoning Genome Chain and Gene Scissors.